Table 3.4 AI providers that implement the language model specification

Provider	Package
OpenAI	`@ai-sdk/openai`
Anthropic	`@ai-sdk/anthropic`
Google generative AI	`@ai-sdk/google`
Google Vertex	`@ai-sdk/google-vertex`
Mistral	`@ai-sdk/mistral`

> **NOTE** This list is not definitive, and it may change, as more providers could be added if they implement language model specification. The SDK authors might also include additional providers in the future.

By "supported providers," I mean that you will be able to use the full spectrum of utilities functions from `generateText`, `streamText`, `generateObject`, and `streamObject`.

Additionally, SDK's language model specification is open source, allowing the community to create custom providers for other AI services. Some notable community-contributed providers include

- LLamaCpp (https://github.com/sgomez/ollama-ai-provider)
- Ollama (https://github.com/nnance/llamacpp-ai-provider)

Both providers showcase an example of the flexibility of having a language model specification to extend the support of multiple AI providers under a common abstraction.

3.3.3 Astra AI project: Integrating multiple AI providers and models

Now that we understand the concepts of language model specification, let's put it into practice. We previously imported the `@ai-sdk/google` to create a Google model when we were integrating streaming functionality. We can now expand our feature set by allowing the client to pass on an AI model preference option when sending the request. For this example, we are going to allow switching between two providers: the existing Google `gemini-2.0-flash` model and the OpenAI `GPT-4` model.

> **NOTE** The source code for the following example is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch03/chat-providers.

Here's what we're going to build:

- A backend system that supports multiple AI providers (OpenAI and Google)
- A frontend that can switch between different models based on user preference
- A user interface that allows model selection and comparison

We start with the backend implementation first.